Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«ИЗУЧЕНИЕ МЕТОДОВ ЗАЩИТЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ»

по дисциплине «Методы защиты программного обеспечения»

Выполнил Волошкевич М.А.
студент гр.5151001/40202 <подпись>

Преподователь / Ерастов В.О.
ассистент <подпись>

Санкт-Петербург
2026г.

1. Цель работы.

Изучение методов защиты программного обеспечения, способов анализа защищённого исполняемого файла и преодоления реализованных механизмов защиты.

2. Задачи

В рамках лабораторной работы необходимо реализовать программу типа crackme, добавить в неё методы защиты, исследовать бинарный файл, выполнить обход отдельных проверок патчингом и изучить упаковку исполняемого файла. В отчёте рассматриваются исходная и защищённая версии программы, назначение внедрённых механизмов и результаты анализа дизассемблированного кода.

3. Ход работы

3.1. Описание исходной программы

Исходная программа написана на языке C. Она читает пароль из файла `password.txt` и сравнивает первые девять символов с заданной строкой `123456789`. При совпадении создаётся серийный номер формата `KEY$xxxxxxxxxx$`, содержащий десять случайных буквенно-цифровых символов, и записывается в `serial.txt`. После успешной проверки программа также принимает целые числа из стандартного ввода и вычисляет элементы последовательности, формируемой умножением предыдущих значений на 3, 5 и 7.

При неверном пароле программа выводит `Wrong pass` и завершает работу без создания серийного номера и выполнения основного цикла вычислений. Файлы `password.txt` и `serial.txt` должны находиться в рабочем каталоге программы.

В исходном варианте проверка пароля непосредственно выражена вызовом `strncmp(pass, "123456789", 9)`. Генерация серийного номера и полезная вычислительная функция `give_n_sequence` не защищены дополнительными проверками целостности или антиотладочными механизмами.

3.2. Основные изменения в защищённой версии

В защищённой версии проверка пароля отделена от места её вызова. Используются функции `check_impl`, `real_check` и `real_check_trampoline`, а функция `patch_check` изменяет машинный код в памяти процесса: в точку `patch_point_impl` записывается относительный переход на trampoline, который передаёт управление `real_check`.

Строковые константы, включая имена файлов, сообщения и части серийного номера, хранятся в XOR-зашифрованном виде и восстанавливаются функцией `xor_str`. В программе присутствуют проверки целостности участка исполняемого кода через CRC32, вызов `ptrace`, проверка временных ограничений и проверка состояния ASLR. Проверки интегрированы в ход выполнения программы, поэтому изменение отдельных переходов может повлиять на дальнейшую работу.

3.3. Анализ проверки пароля и бинарный патчинг

В предоставленном дизассемблерном фрагменте вызов `strncmp` расположен по адресу `0x555555555B3D`, а после него выполняется `test eax, eax`. Следующий условный переход в оригинальном потоке управления определяет, равны ли сравниваемые строки. В исследуемом варианте переход заменён безусловным `jmp` на адрес `0x555555555B50`.

Такое изменение заставляет выполнение продолжаться по ветви, предназначенной для успешного результата проверки, независимо от значения, возвращённого `strncmp`. В отчёте это рассматривается как пример патчинга условного перехода в дизассемблере. Для фиксации эксперимента следует приложить скриншот исходного перехода и изменённой инструкции, а также подтвердить запуском программы, что после патчинга действительно создаётся `serial.txt`.



Рис. 1. Проверка пароля в дизассемблере до и после патчинга

3.4. Анализ проверок целостности и среды выполнения

В листинге `main` вычисляется CRC32 для диапазона адресов между метками `loc_555555555DBA` и `loc_555555556765`. Полученное значение сравнивается с константой `0xC8ADB734`. В исходном фрагменте условный переход после `cmp` заменён на безусловный `jmp`, благодаря чему программа переходит к продолжению выполнения, минуя ветвь вывода предупреждения `[ALERT]`.

Далее вызывается `ptrace`, после чего возвращаемое значение сравнивается с `-1`. В предоставленном листинге переход после сравнения заменён на безусловный переход к дальнейшему коду. Аналогично показаны изменения ветвлений для проверки временного ограничения и проверки флага

ASLR. Эти фрагменты демонстрируют, что проверка защиты может быть найдена по вызову системной функции, сравнению результата и условному переходу, ведущему к диагностическому сообщению или завершению программы.

В другом участке кода проверка `check_impl` выполняется перед созданием и записью серийного номера, а также в цикле программы. Это позволяет проверять условие не только в единственной начальной точке, но и повторно во время работы.



Рис. 2. Фрагменты дизассемблированного кода: CRC32, ptrace и проверки среды

3.5. Обфускация строк и изменение потока управления

XOR-шифрование строк скрывает в секции данных исходные текстовые значения. Перед использованием соответствующая строка восстанавливается функцией `xor_str`; поэтому поиск обычных строковых констант в бинарном файле может не дать ожидаемого результата. При анализе необходимо проследить аргументы вызова функции расшифрования и момент, когда восстановленная строка передаётся библиотечной функции.

В защищённой версии присутствуют несколько точек вызова `check_impl`, а также переходы, изменённые патчингом. Такой подход усложняет анализ по одному участку кода, поскольку необходимо учитывать все места проверки и связи между ними. Самомодификация реализована через изменение прав доступа страницы памяти с `mprotect` и запись машинных байтов в `patch_point_impl`; затем права страницы возвращаются к режиму чтения и исполнения.

Требование методички о трёх методах противодействия дизассемблированию, четырёх методах обнаружения отладчика и трёх методах обнаружения виртуальной машины следует подтверждать конкретными реализациями в итоговом исходном коде. По предоставленным фрагментам можно непосредственно подтвердить самомодификацию, XOR-обфускацию строк, CRC-проверку, `ptrace`, проверку времени и проверку ASLR; наличие остальных методов нельзя считать доказанным только по приведённым листингам.

3.6. Упаковка исполняемого файла

Для упаковки использовался UPX. В предоставленном выводе каталога присутствуют исходный исполняемый файл `main` размером около 23 KiB и упакованный `main_upx` размером около 12 KiB. По этим округлённым значениям уменьшение размера составляет приблизительно 47,8 %. Это оценка по округлённому выводу `ls -lh`, а не точное измерение размера в байтах.

UPX добавляет загрузочный код, который восстанавливает упакованные данные в памяти и передаёт управление оригинальной точке входа (OEP). Точное значение OEP следует определить при анализе конкретного `main_upx` в дизассемблере или отладчике. По одному имени файла и приведённым фрагментам защищённого бинарника достоверно установить адрес OEP нельзя.

Для сравнения энтропии необходимо измерить оба бинарных файла, а не оценивать значение по размеру или листингу. Энтропия Шеннона для байтов вычисляется по формуле:

$$H(X) = -\sum_{i=0}^{255} p_i \log_2(p_i)$$

где p_i — доля байта со значением i в файле. Максимальное значение составляет 8 бит на байт. Упаковка часто повышает энтропию упакованной области, однако фактические значения для `main` и `main_upx` нужно получить измерением на исходных файлах.

Параметр	**main**	**main_upx**
Размер по `ls -lh`	около 23 KiB	около 12 KiB
Размер в байтах	измерить	измерить

Энтропия, бит/байт	измерить	измерить
Точка входа	из `readelf`	исследовать OEP



Рис. 3. Сравнение характеристик исходного и упакованного файлов

3.7. Что необходимо подтвердить экспериментально

В представленном материале есть исходный код до защиты, защищённый исходный файл и фрагменты дизассемблирования с изменёнными переходами. В отчёте не приводятся как измеренные факты те результаты, для которых нет полного бинарного файла или протокола запуска: точные значения энтропии, адрес OEP, успешность обхода всех защит и фактическое создание серийного номера после патчинга. Эти значения необходимо добавить после повторения экспериментов на локальных файлах.

Для получения точных размеров можно использовать `stat -c '%n %s' main main_upx`, а для первичного анализа ELF — `readelf -h main` и `readelf -h main_upx`. Энтропию можно измерить локальным инструментом анализа файлов или небольшим скриптом, считающим частоты всех 256 значений байта. Полученные значения следует внести в таблицу выше.

4. Выводы

В ходе работы рассмотрена программа на C, выполняющая проверку пароля, создание серийного номера и вычисление числовой последовательности. Исследованы способы изменения поведения бинарного файла путём замены условных переходов, а также механизмы защиты, присутствующие в защищённой версии: XOR-обфускация строк, CRC32-проверка участка

кода, вызов `ptrace`, проверка времени и состояния ASLR, повторные проверки и самомодификация через `mprotect`.

По предоставленным дизассемблерным фрагментам показано, как обнаружить сравнение пароля, найти связанные с проверками ветви отказа и изменить управление выполнением. Также рассмотрена цель упаковки UPX и необходимость сравнивать исходный и упакованный файлы по точным размерам, энтропии и точке входа. Для завершения отчёта требуется добавить результаты измерений и скриншоты, полученные при запуске инструментов на фактических бинарных файлах.

Приложения

Приложение А. Исходный код до внедрения защиты

`main_simple.c`

Приложение Б. Исходный код защищённой версии

main.c